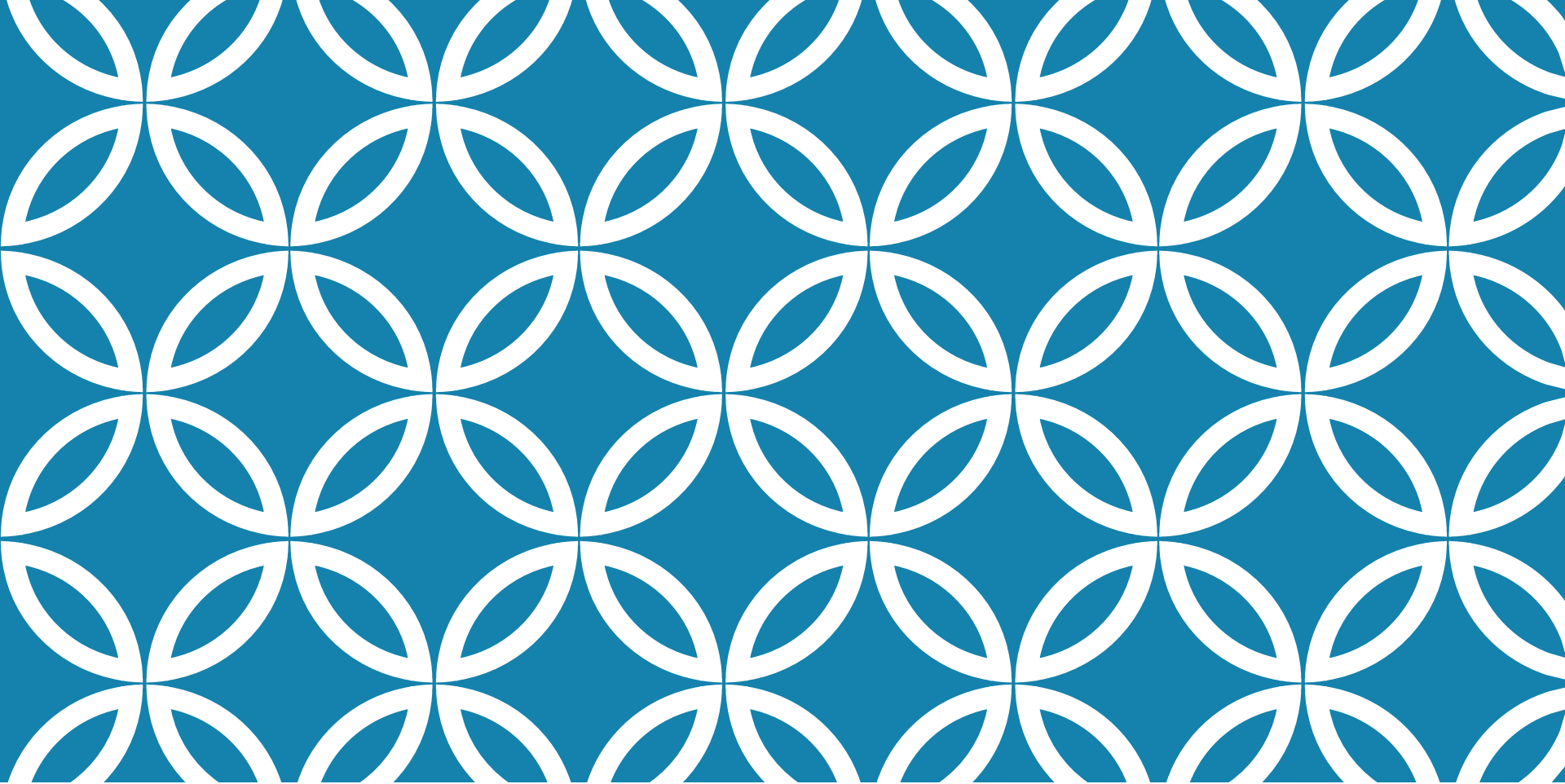




LAYERED, PIPELINE ARCHITECTURAL STYLES

Lecture 4

LAST TIME

- Architectural characteristics
- Component-based thinking

CONTENT

Architectural Styles

- Foundations
- Layered
 - MVC and variants
- Pipeline

REFERENCES

Fundamentals of Software Architecture, Mark Richards,
Neal Ford, 2020 - Chapters 9, 10, 11

ARCHITECTURAL STYLE

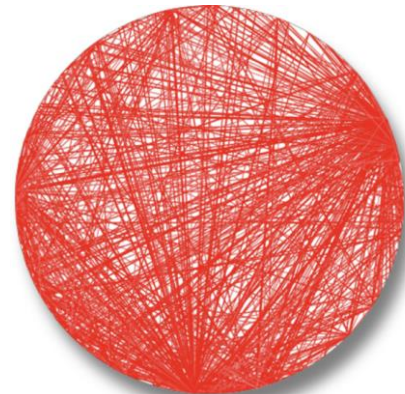
- Architectural style = structure of how the user interface and backend source code are organized and how that source code interacts with a data store.
- Each style has advantages and limitations => challenge is to make decisions for the particular problem

FUNDAMENTAL STYLES

- Big ball of mud [Foote&Yoder, 1997] = no discernible architecture

Example: simple scripting application with event handlers that access databases directly.

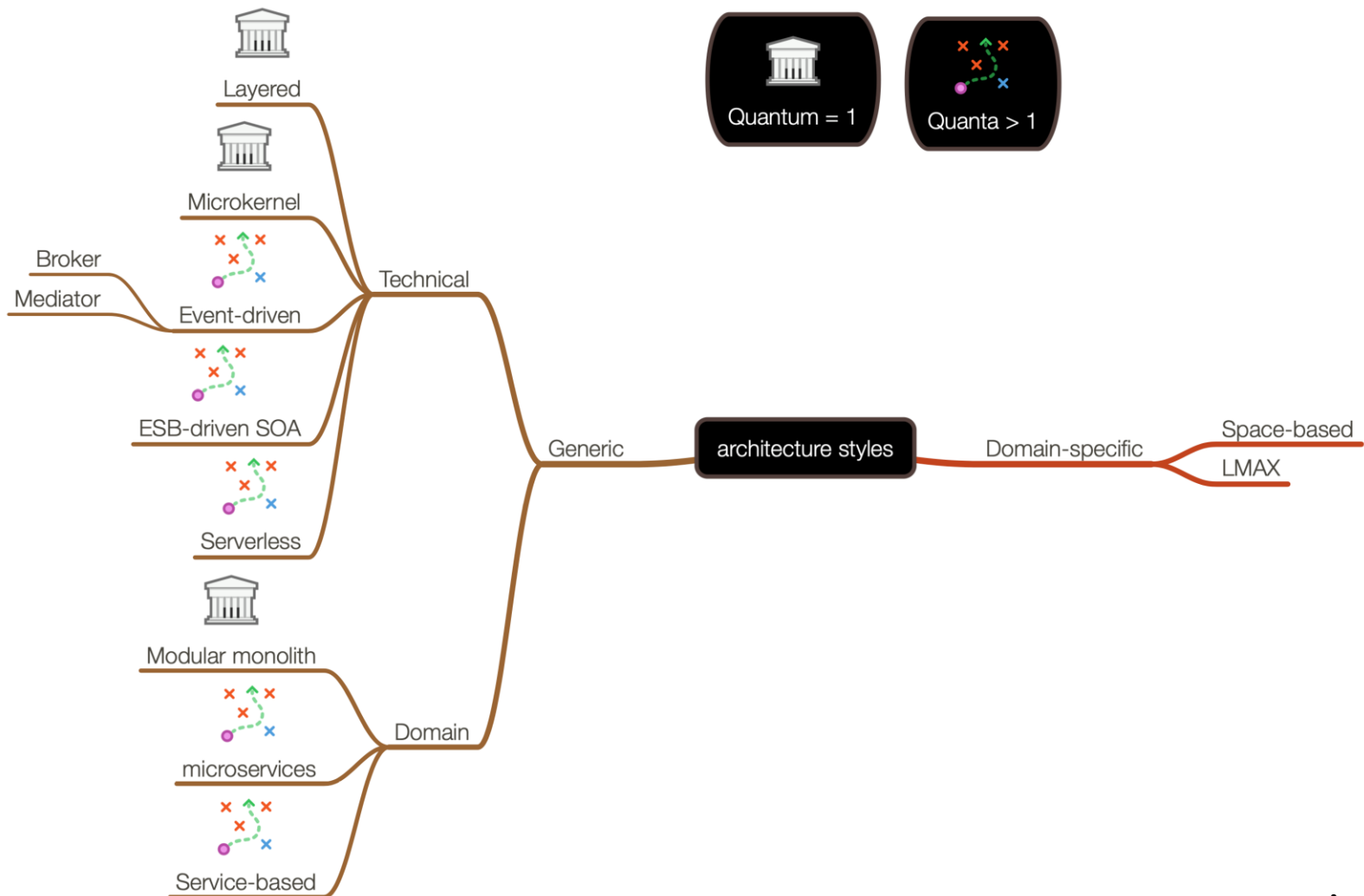
⇒ Problems in maintainability, deployment, testability, scalability and performance.



MONOLITHIC VS DISTRIBUTED ARCHITECTURE

- Monolithic = single deployment unit of all code (1 architectural quantum)
 - Layered
 - Pipeline
 - Microkernel
- Distributed = multiple deployment units connected through remote access protocols (several architectural quanta)
 - Service-based
 - Event-driven
 - Service-oriented
 - Microservices

MONOLITHIC VS DISTRIBUTED ARCHITECTURE

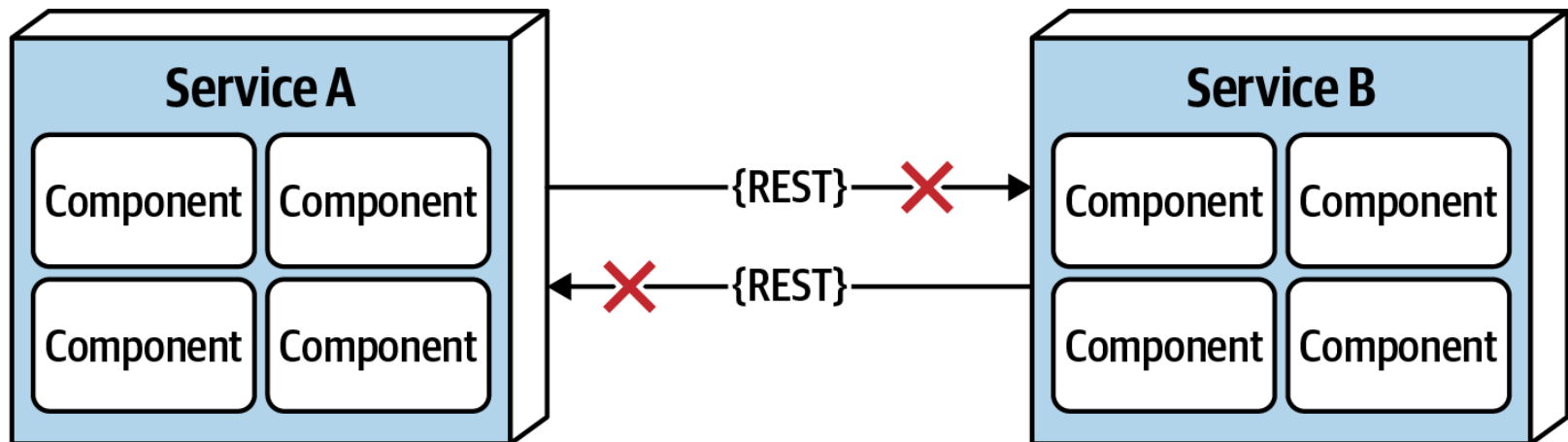


DISTRIBUTED ARCHITECTURE

- Much more powerful in terms of performance, scalability, and availability
- What can go wrong?

Fallacy #1: The Network is reliable

⇒ The more a system relies on the network (such as microservices architecture), the potentially less reliable it becomes.

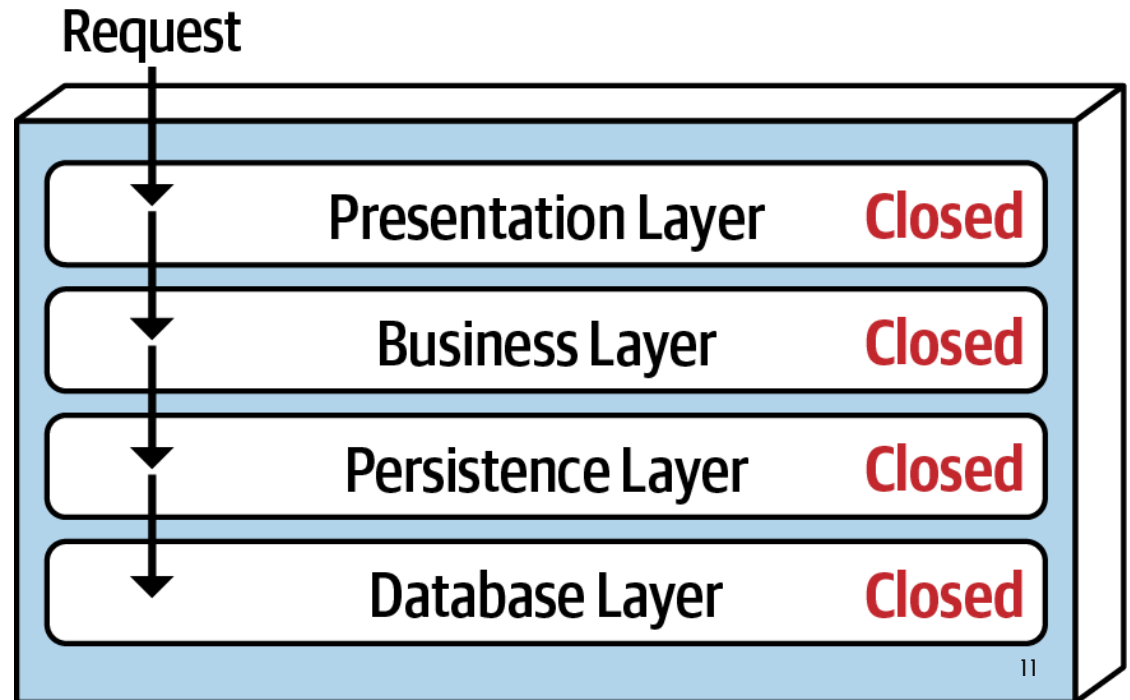


DISTRIBUTED ARCHITECTURE FALLACIES

- Homework = read about the Distributed architecture fallacies in [1], Chapter 9.

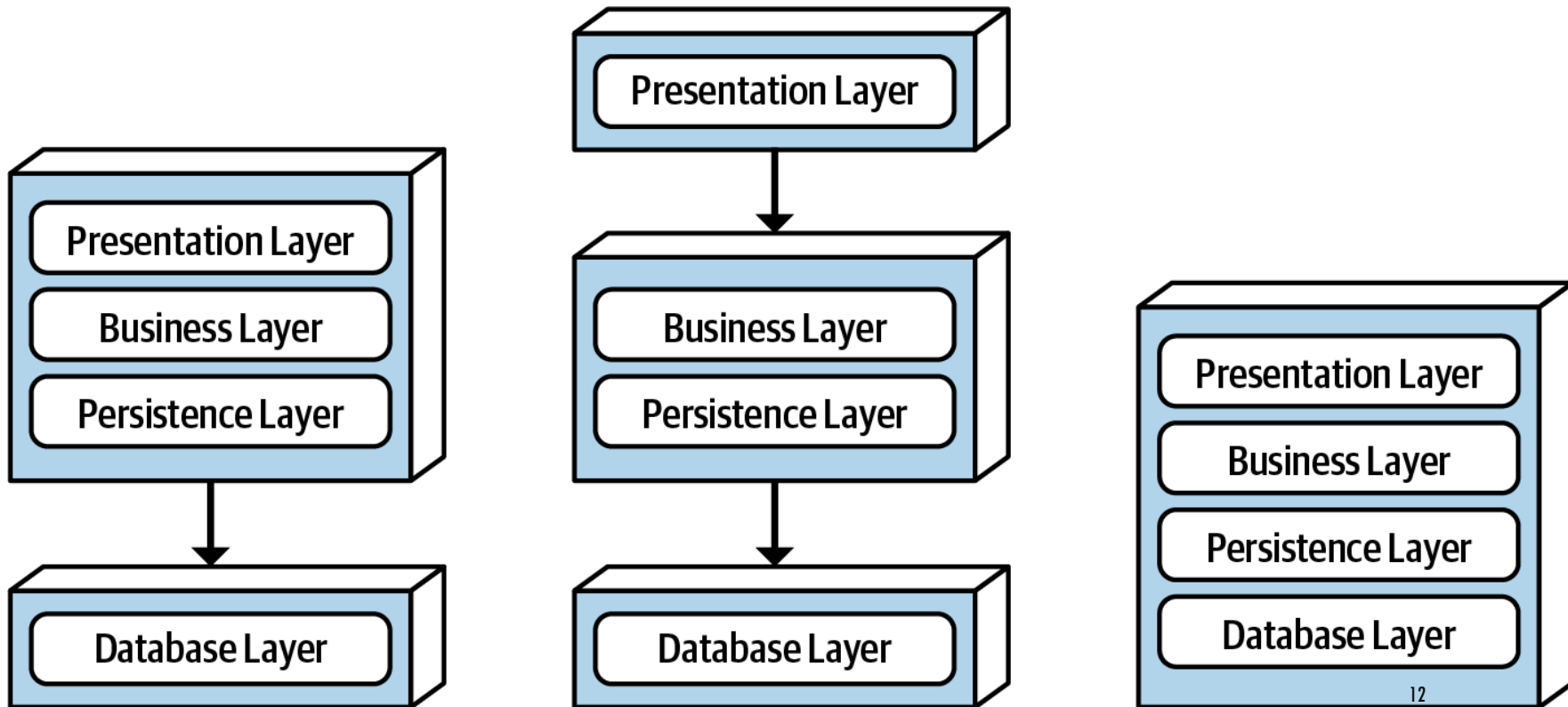
LAYERED ARCHITECTURE

- **Technically** partitioned architecture (separation of concerns)
- Layers of isolation = changes made in one layer of the architecture generally don't impact or affect components in other layers



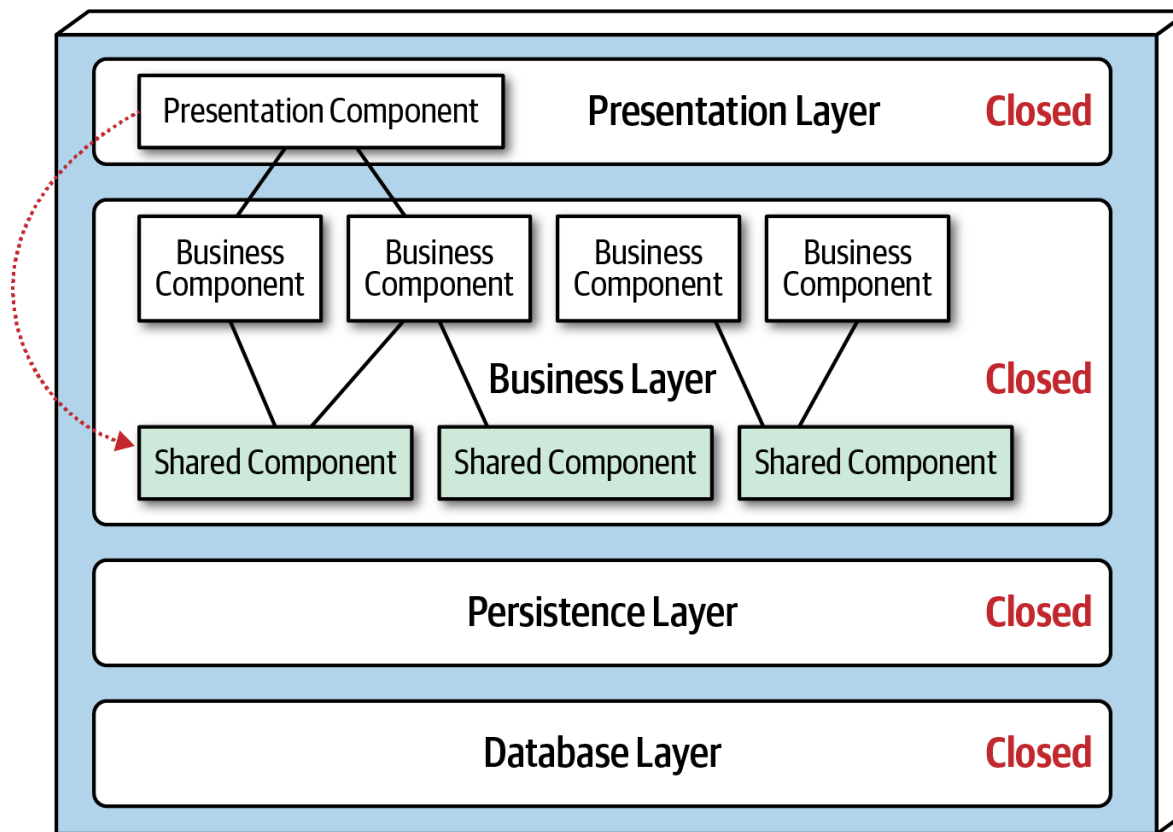
DEPLOYMENT PERSPECTIVE

- Logic layers can be grouped in various configurations of deployment units (i.e. physical topology)



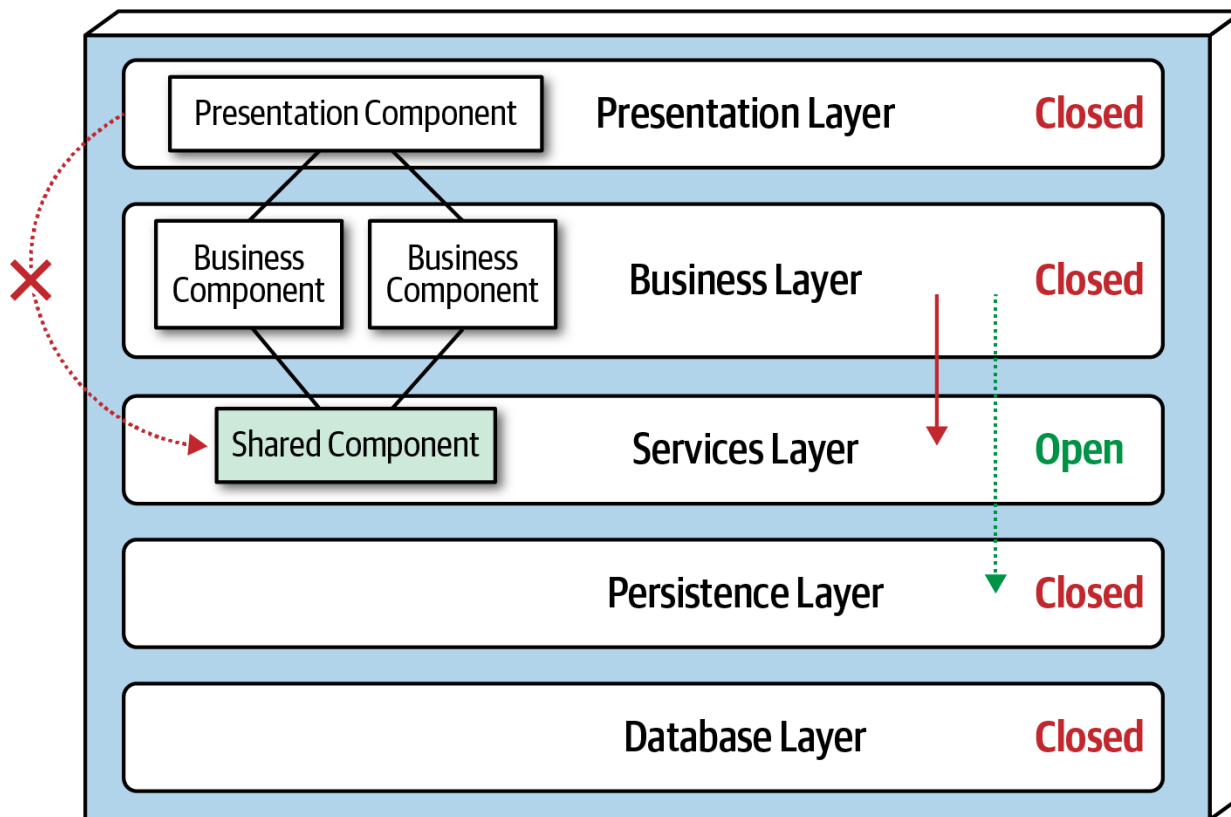
ADDING LAYERS

- Why?
 - If you want to prevent certain components from being accessed



OPENING LAYERS

- Why?
 - If you want to allow access to non-adjacent layers



OTHER CONSIDERATIONS

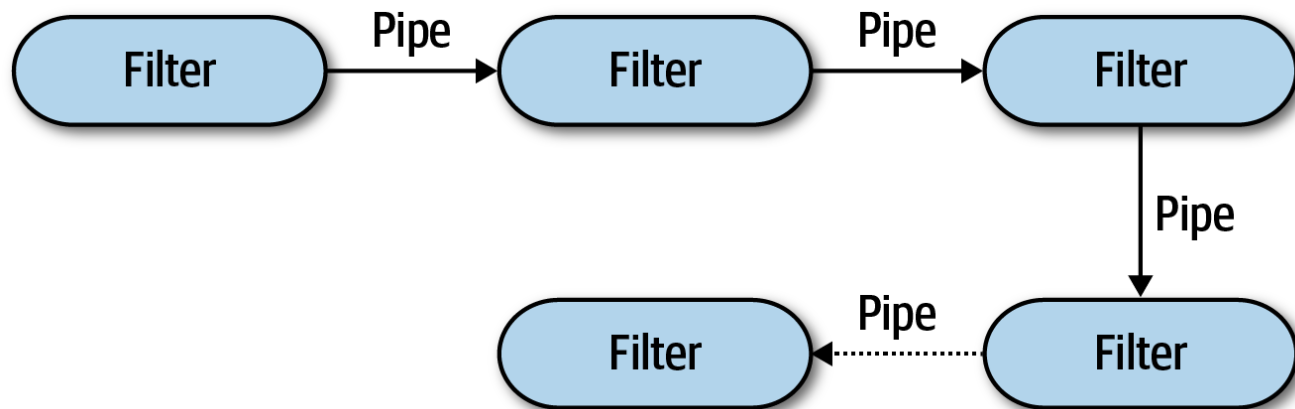
- Layered architecture is a good starting point
- Good choice for small, simple applications
- One of the lowest-cost architecture styles, promoting ease of development
- Avoid architecture **sinkhole anti-pattern!**
 - when requests move from layer to layer as simple pass-through processing with no business logic performed within each layer

ARCHITECTURE CHARACTERISTICS RATINGS

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★

PIPELINE ARCHITECTURE

- Topology: pipes and filters (sequential processing)
- Pipes = communication channel between filters, unidirectional and point-to-point
- Filters = self-contained, independent from other filters, and generally stateless processing units that perform one task only



FILTERS

- **Producer**

- The starting point of a process, outbound only, sometimes called the source.

- **Transformer**

- Accepts input, optionally performs a transformation on some or all of the data, then forwards it to the outbound pipe.

- **Tester**

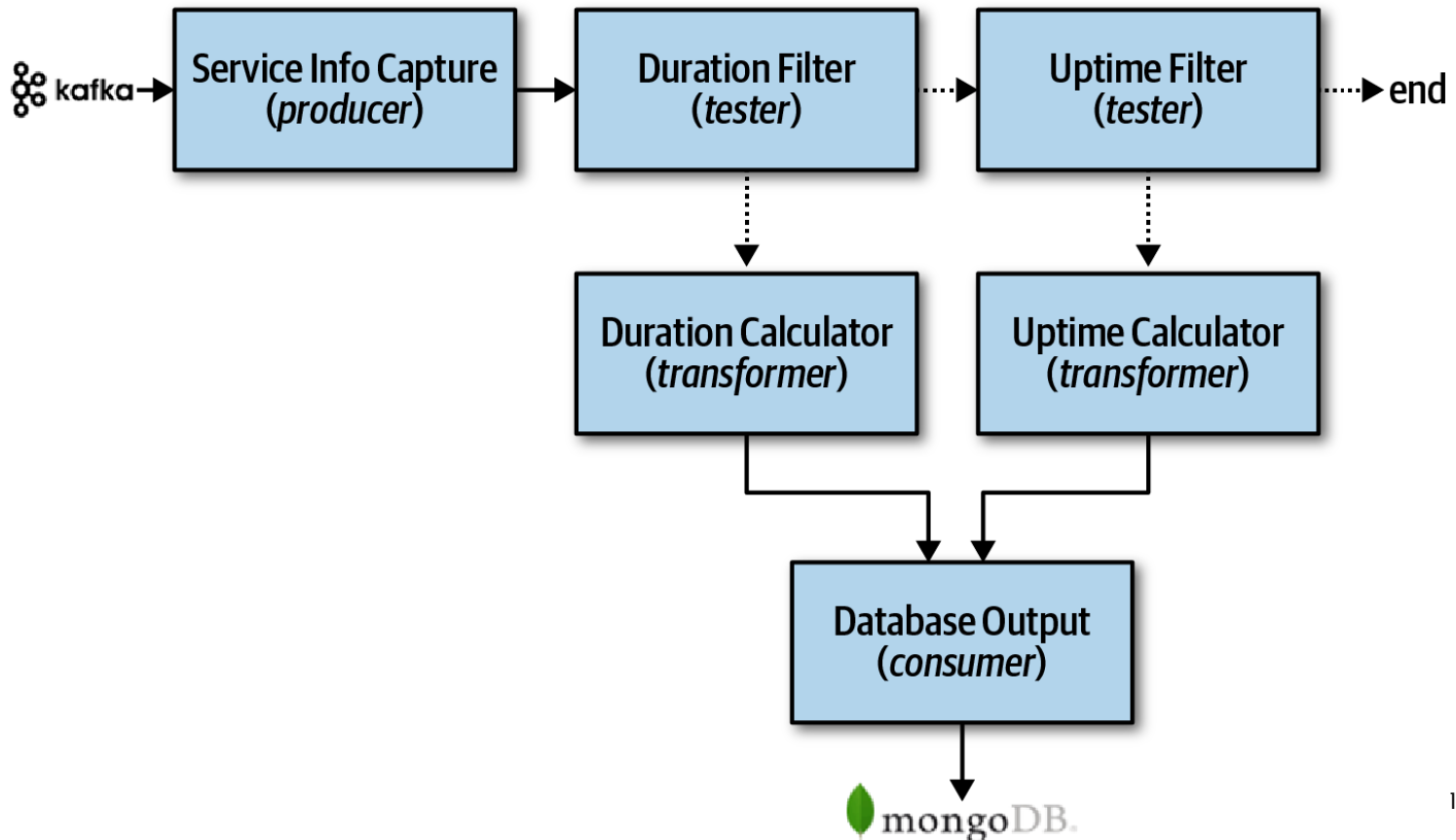
- Accepts input, tests one or more criteria, then optionally produces output, based on the test.

- **Consumer**

- The termination point for the pipeline flow. Responsibility: **persist** the final result of the pipeline process to a database and/or **display** the final results on a user interface screen.

EXAMPLE

- various service telemetry information is sent from services via streaming to Apache Kafka.



ARCHITECTURE CHARACTERISTICS RATINGS

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★ ★
Elasticity	★
Evolutionary	★ ★ ★
Fault tolerance	★
Modularity	★ ★ ★
Overall cost	★ ★ ★ ★ ★
Performance	★ ★
Reliability	★ ★ ★
Scalability	★
Simplicity	★ ★ ★ ★ ★
Testability	★ ★ ★

MODEL VIEW CONTROLLER (MVC)

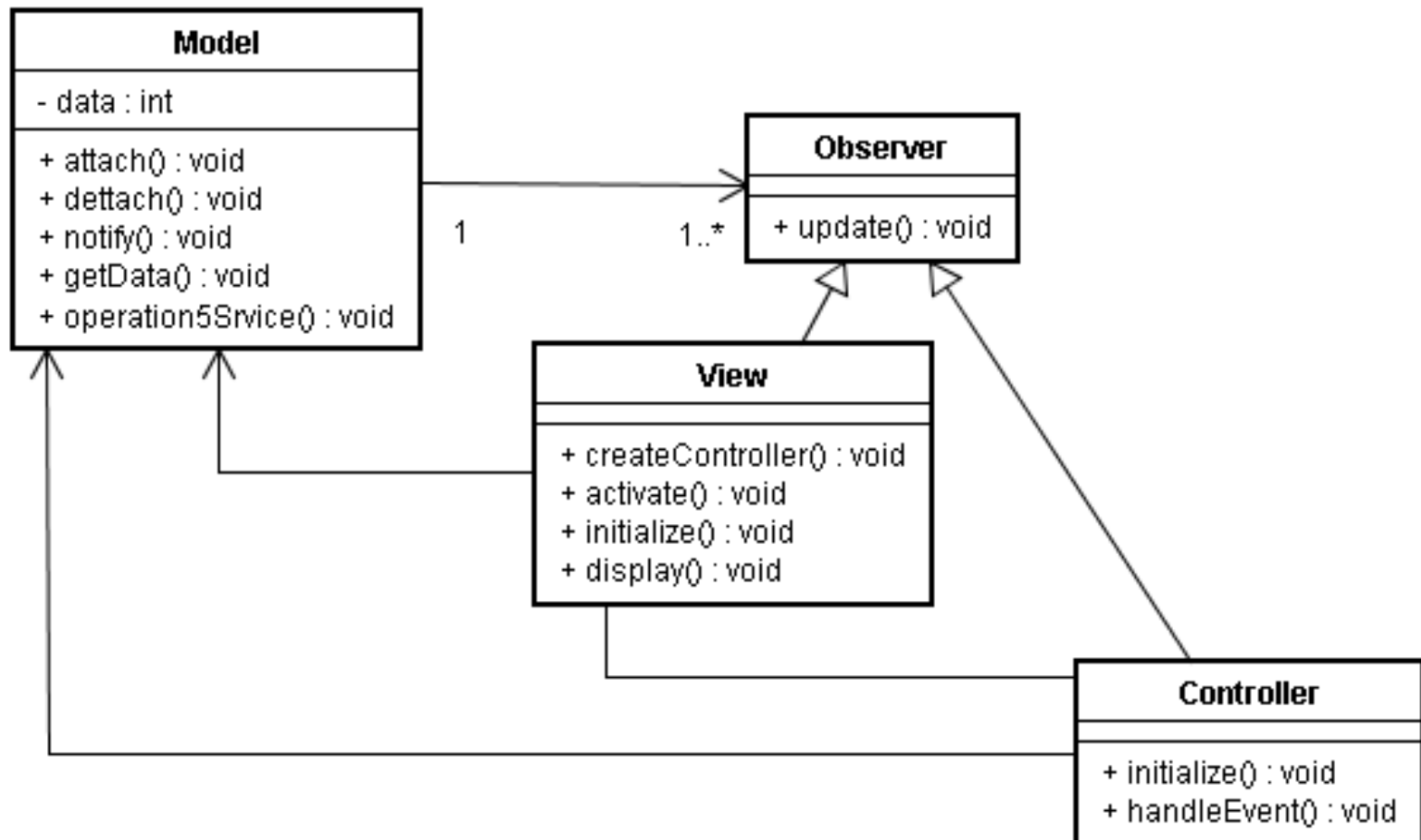
Solution

- 3 areas: handle input, processing, output
- The **Model** component encapsulates core data and functionality (processing).
- **View** components display information to the user. A view obtains the data from the model (output).
- Each view has an associated **Controller** component. Controllers handle input.

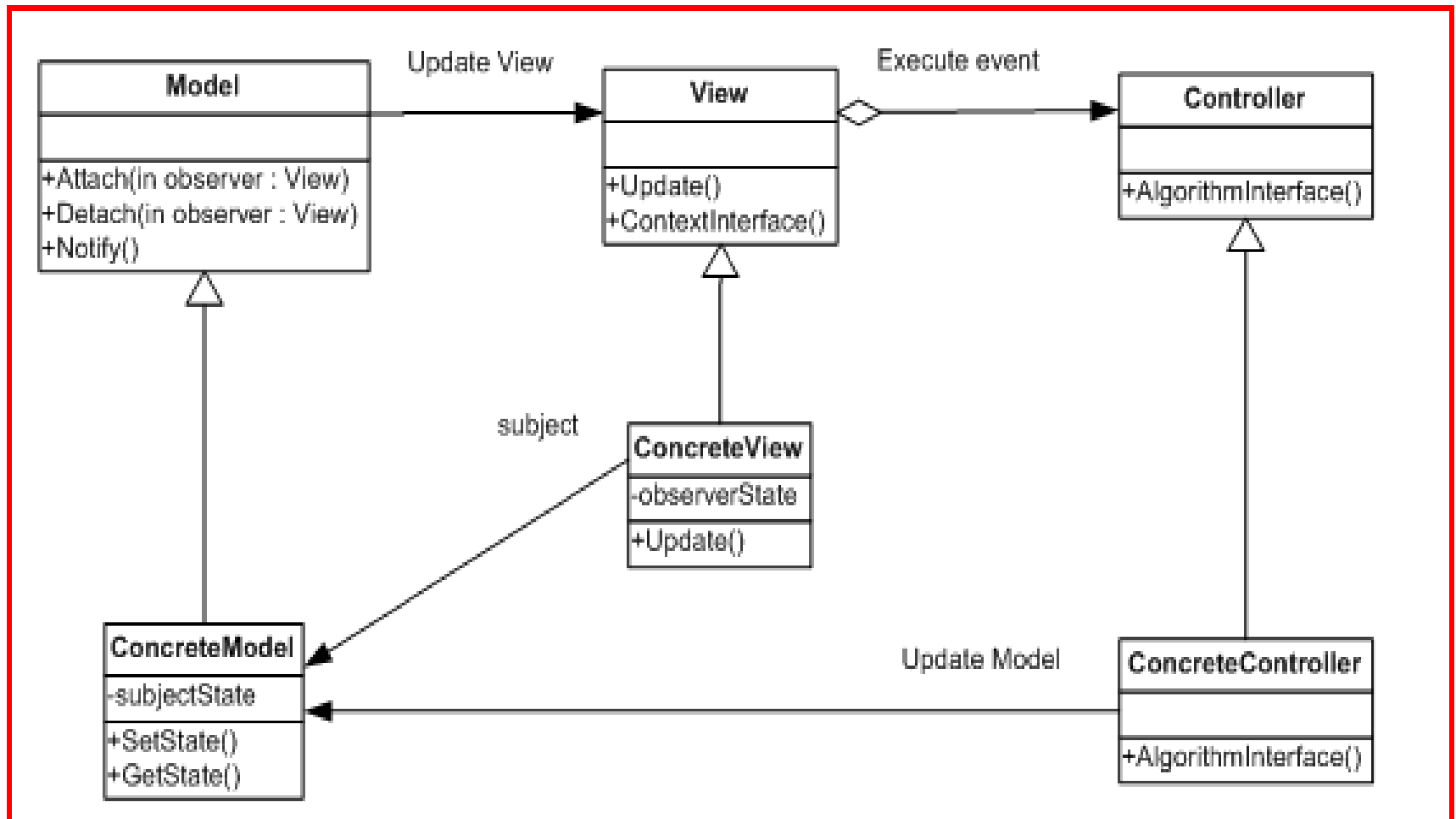
CONSTRAINTS

- The **same information is presented differently** in different windows, for example, in a bar or pie chart.
- The display and behavior of the application must **reflect data manipulations immediately**.
- **Changes to the user interface** should be easy, and even possible **at run-time**.
- Supporting **different 'look and feel' standards** or porting the user interface should not affect code in the core of the application.

MVC STRUCTURE



MVC MORE DETAILED



THE MODEL

- encapsulates and manipulates the **domain data** to be rendered
- has **no** idea how to **display** the information is has **nor does it interact** with the user or receive any user input
- encapsulates the functionality necessary to manipulate, obtain, and deliver that data to others, *independent* of any user interface or any user input device

THE VIEW

- Is a **specific visual rendering** of the information contained in the model (graphical, text-based).
- **Multiple views** may present multiple renditions of the data in the model
- Each view is ***dependent*** of a model
- When the model changes, all dependent views are updated

THE CONTROLLER

- handles user input. They “listen” for user direction, and *handle* requests using the model and views
- often watches mouse events and keyboard events
- allows the decoupling of the model and its views, allowing views to simply render data and models to simply encapsulate data
- is “paired up” with collections of view types, so that a “pie graph” view would be associated with its own “pie graph” controller, etc.
- **The *behavior* of the controller is dependent upon the *state* of the model**

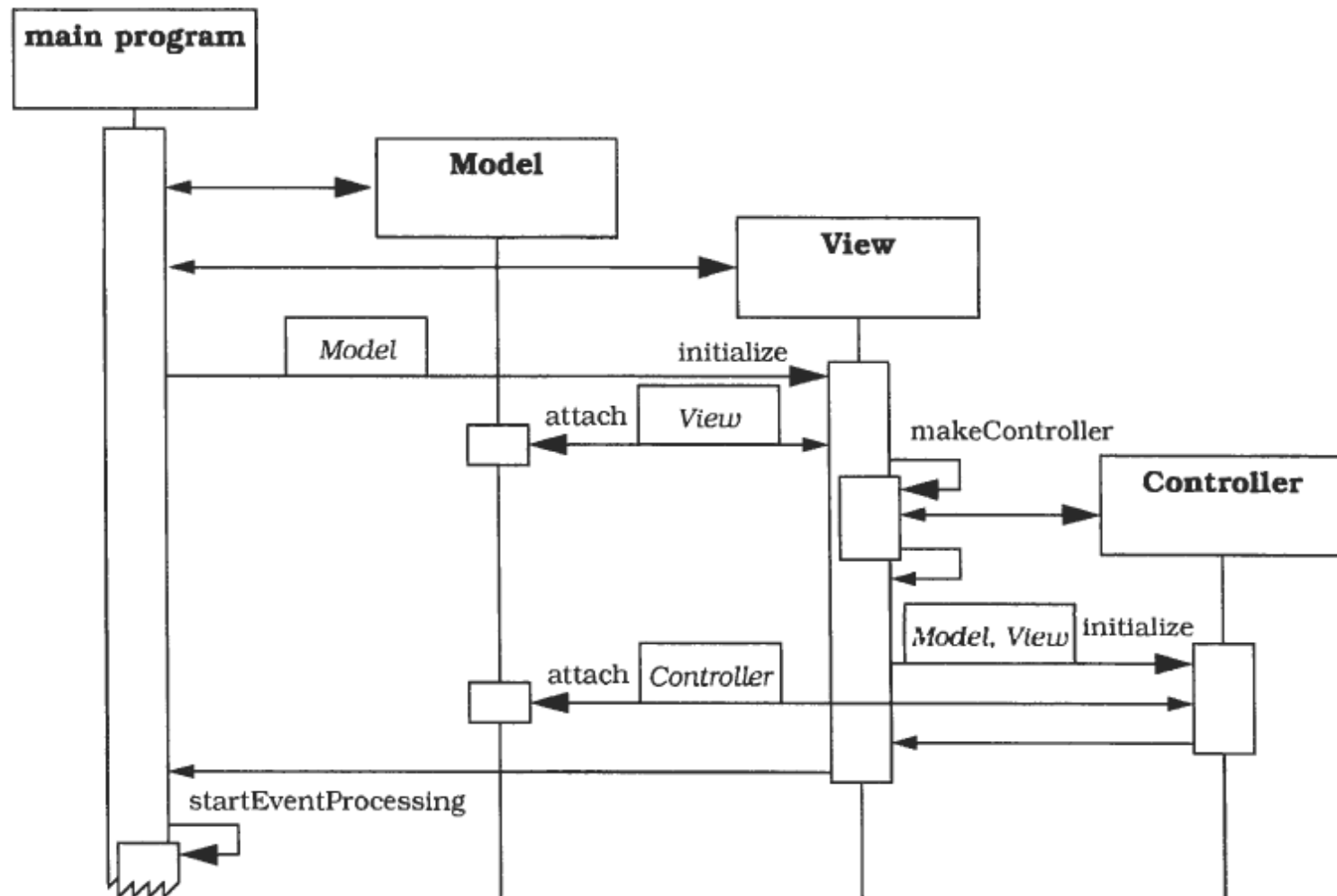
HOW DOES THIS WORK?

- The **model** has a list of **views** it supports
- Each **view** has a reference to its **model**, as well as its **supporting controller**
- Each **controller** has a reference to the **view** it controls, as well as **to the model** the view is based on. However, models know nothing about controllers.
- On user input, the controller notifies the model which in turn notifies its views of a change

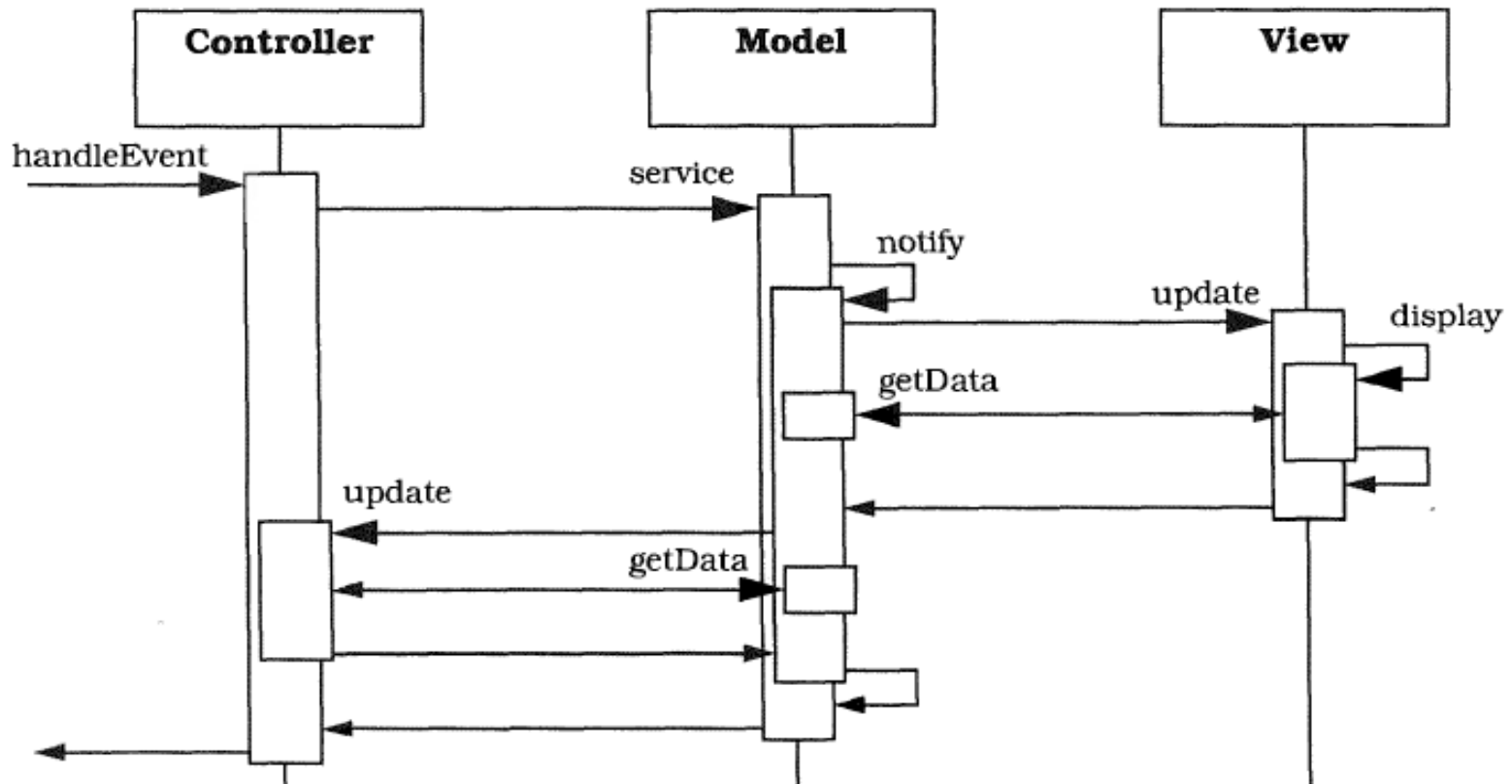
Changing a controller's view will give a different *look*.

Changing a view's controller will give a different *feel*.

SCENARIO I – CREATING THE M,V,C OBJECTS



SCENARIO II – EVENT HANDLING



CONSEQUENCES

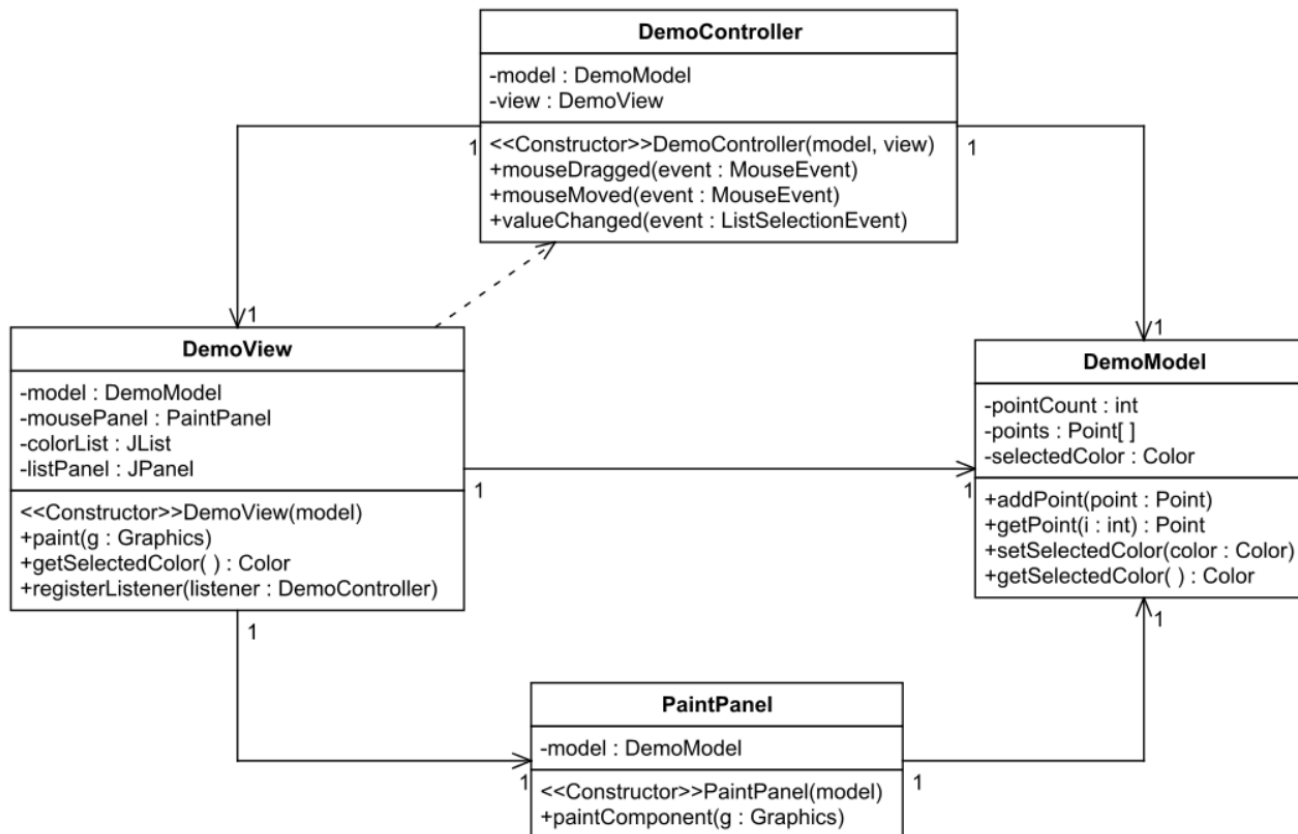
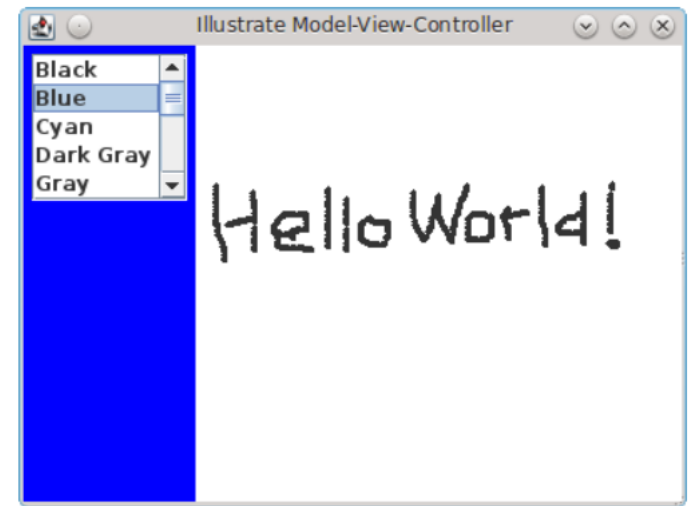
Benefits

- Multiple views of the same model.
- Synchronized views.
- 'Pluggable' views and controllers
- Exchangeability of 'look and feel'.
- Framework potential.

Liabilities

- Increased complexity.
- Potential for excessive number of updates.
- Intimate connection between view and controller.
- Close coupling of views and controllers to a model
- Inefficiency of data access in view.
- Inevitability of change to view and controller when porting.

MVC EXAMPLE



MODEL VIEW PRESENTER

Intent

- Separation between
 - Data
 - Business Logic
 - UI
- Enforce single responsibility: M-V-P
- Reduce coupling
- Facilitate isolated testing

MVP STRUCTURE

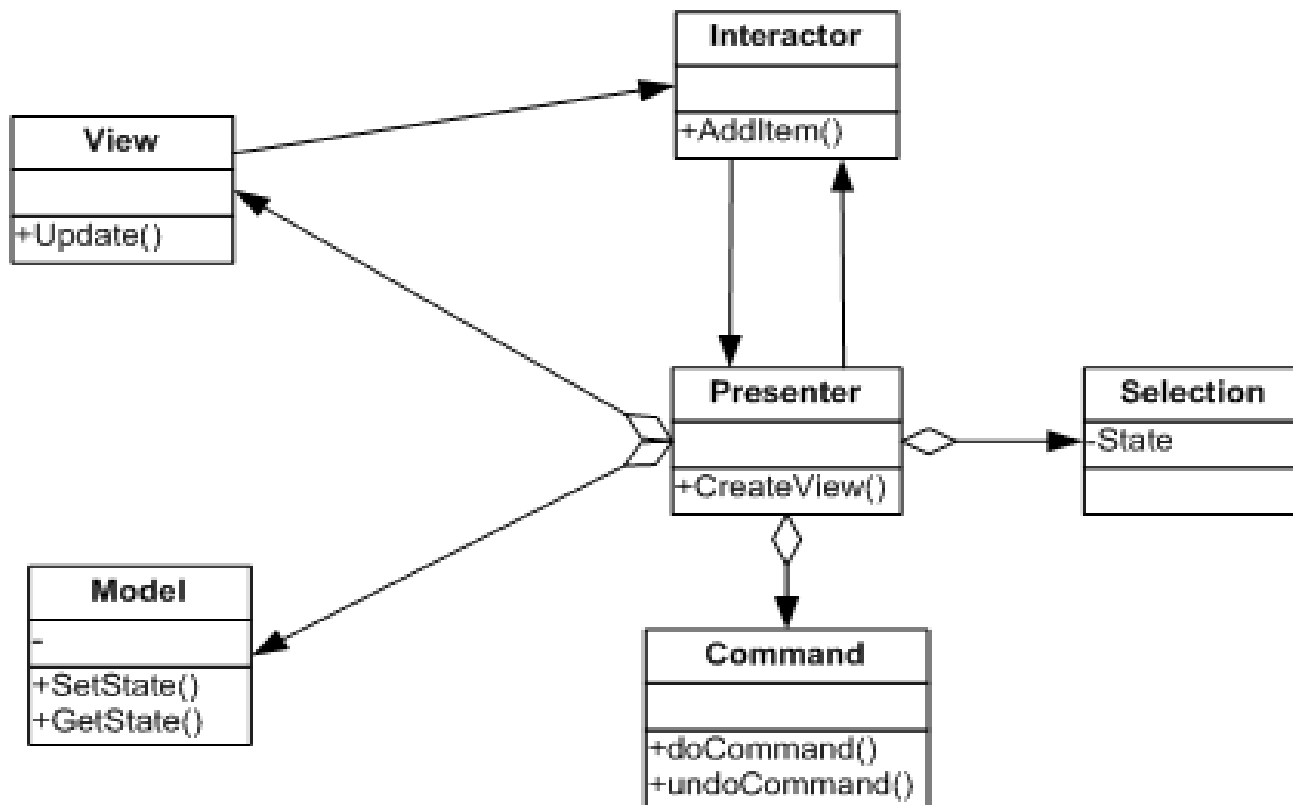
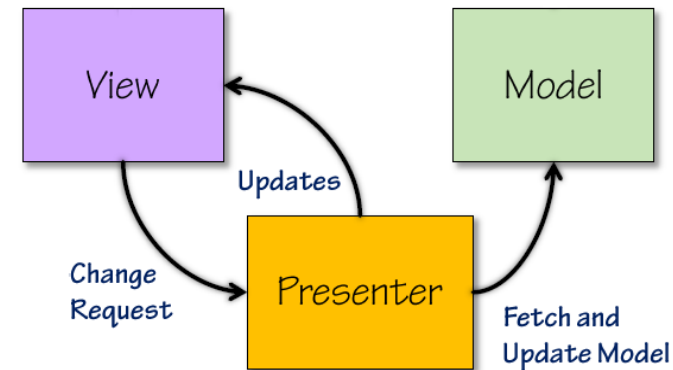


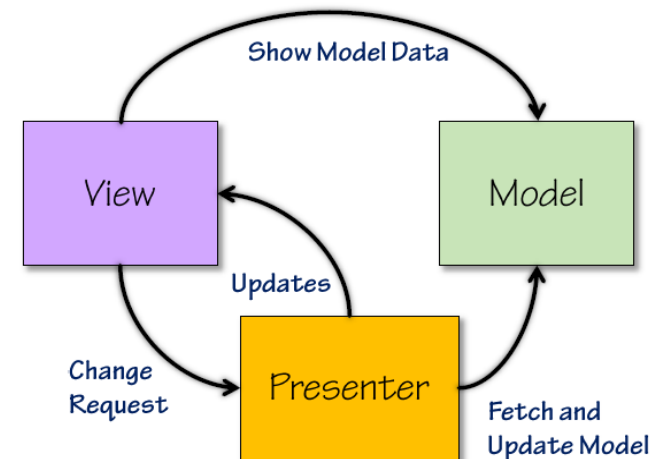
Figure 6: MVP

VARIANTS

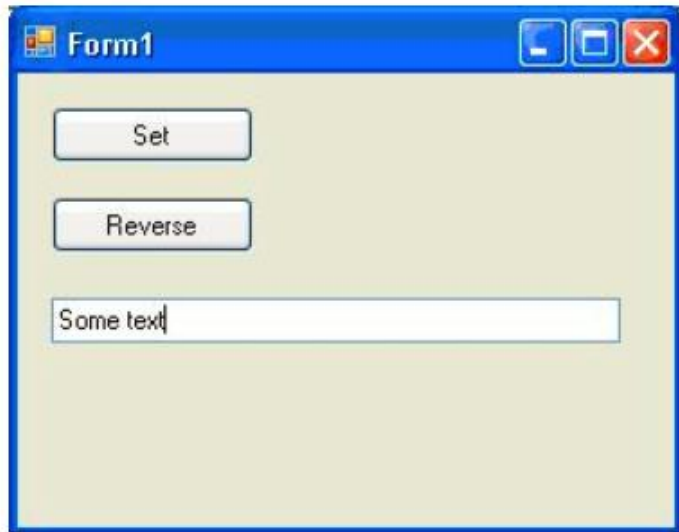
- View and Model are decoupled
- Presenter coordinates events between View and Model
- Passive View



- View and Model no longer separated
- Presenter coordinates events between View and Model
- Model contains only what View needs
- Presenter interacts with Domain Layer/Data Access



MVP EXAMPLE



```
namespace ModelViewPresenter
{
    public partial class Form1 : Form, IView
    {
        private Presenter presenter = null;
        private readonly Model m_Model;

        public Form1(Model model)
        {
            m_Model = model;
            InitializeComponent();
            presenter = new Presenter(this, m_Model);
            SubscribeToModelEvents();
        }

        public string TextValue
        {
            get
            {
                return textBox1.Text;
            }
            set
            {
                textBox1.Text = value;
            }
        }

        private void Set_Click(object sender, EventArgs e)
        {
            presenter.SetTextValue();
        }

        private void Reverse_Click(object sender, EventArgs e)
        {
            presenter.ReverseTextValue();
        }

        private void SubscribeToModelEvents()
        {
            m_Model.TextSet += m_Model_TextSet;
        }

        void m_Model_TextSet(object sender, CustomArgs e)
        {
            this.textBox1.Text = e.m_after;
            this.label1.Text = "Text changed from " + e.m_before + " to " + e.m_after;
        }
    }
}
```

Model
injection in
constructor

Creates
Presenter

View and
Model
injected

```
namespace ModelViewPresenter
{
    public class Presenter
    {
        private readonly IView m_View;
        private IModel m_Model;

        public Presenter(IView view, IModel model)
        {
            this.m_View = view;
            this.m_Model = model;
        }

        public void ReverseTextValue()
        {
            string reversed = ReverseString(m_View.TextValue);
            m_Model.Reverse(reversed);
        }

        public void SetTextValue()
        {
            m_Model.Set(m_View.TextValue);
        }

        private static string ReverseString(string s)
        {
            char[] arr = s.ToCharArray();
            Array.Reverse(arr);
            return new string(arr);
        }
    }
}
```

```
namespace ModelViewPresenter
{
    public class Model : IModel
    {
        private string m_textValue;

        public event EventHandler<CustomArgs> TextSet;
        public event EventHandler<CustomArgs> TextReverse;

        public Model()
        {
            m_textValue = "";
        }

        public void Set(string value)
        {
            string before = m_textValue;
            m_textValue = value;
            RaiseTextSetEvent(before, m_textValue);
        }

        public void Reverse(string value)
        {
            string before = m_textValue;
            m_textValue = value;
            RaiseTextSetEvent(before, m_textValue);
        }

        public void RaiseTextSetEvent(string before, string after)
        {
            TextSet(this, new CustomArgs(before, after));
        }
    }

    public class CustomArgs : EventArgs
    {
        public string m_before { get; set; }
        public string m_after { get; set; }

        public CustomArgs(string before, string after)
        {
            m_before = before;
            m_after = after;
        }
    }
}
```

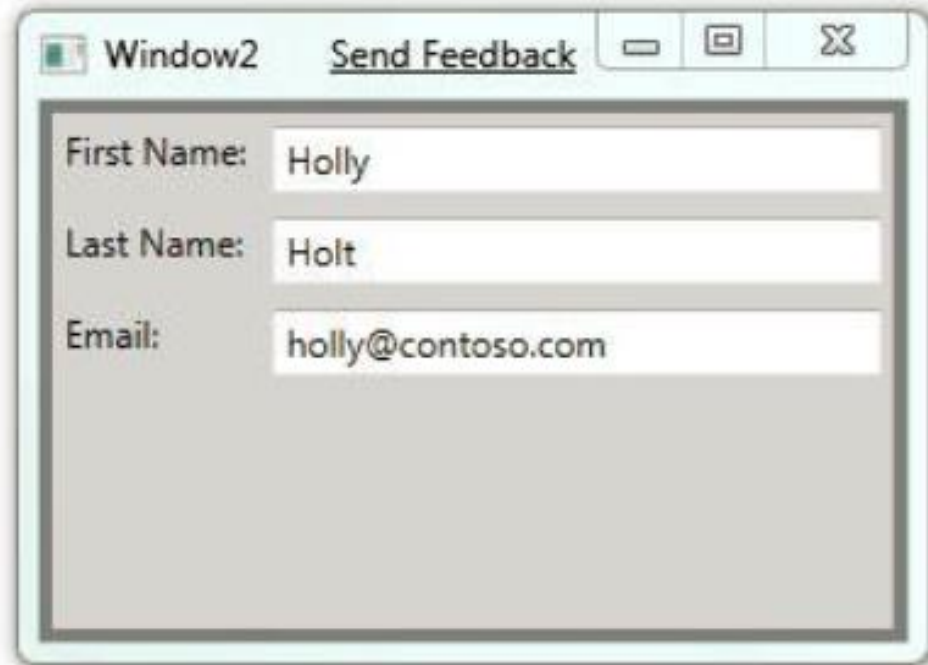
MVVM

Motivation

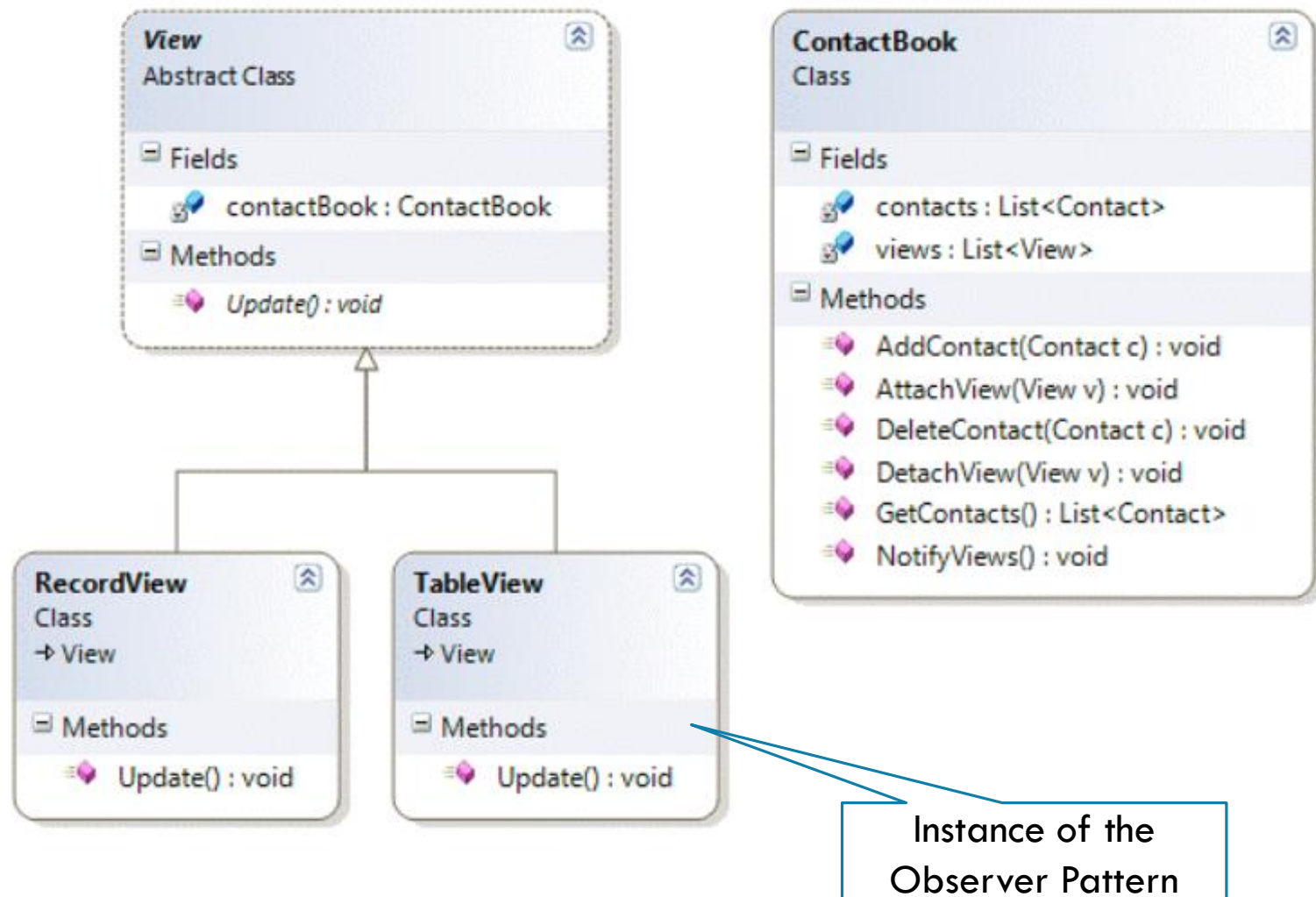
- Need to share a project with a designer, and flexibility for design work and development work to happen near-simultaneously
- Thorough unit testing for your solutions
- Important to have reusable components, both within and across projects
- Flexibility to change the user interface without having to refactor other logic in the code base

MV* EXAMPLE

UI Contacts App



M-V SEPARATION



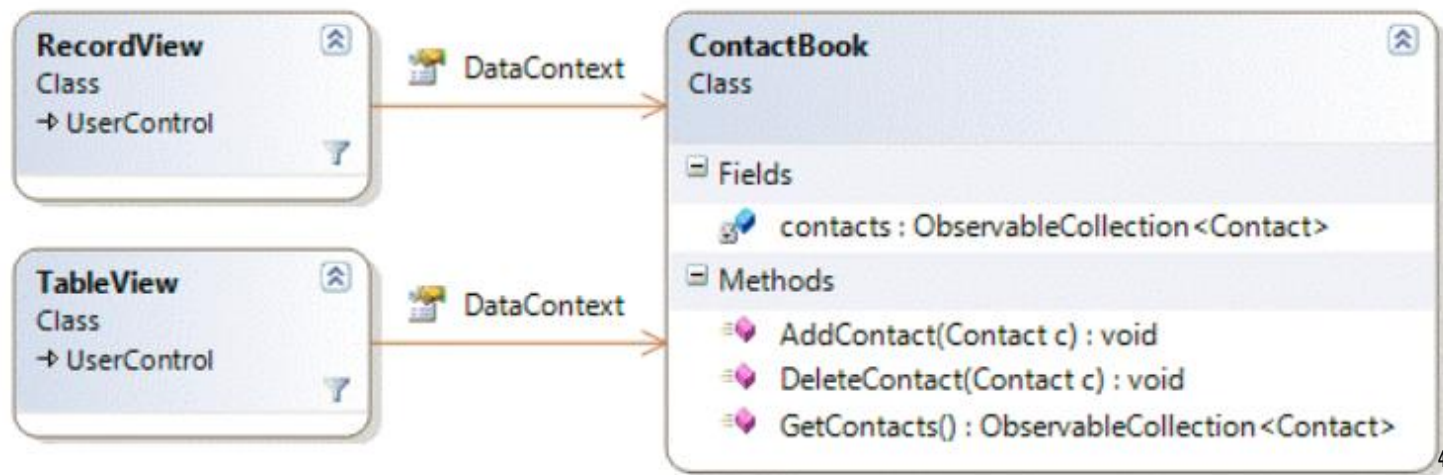
SIMPLISTIC WPF /SILVERLIGHT DESIGN

New features:

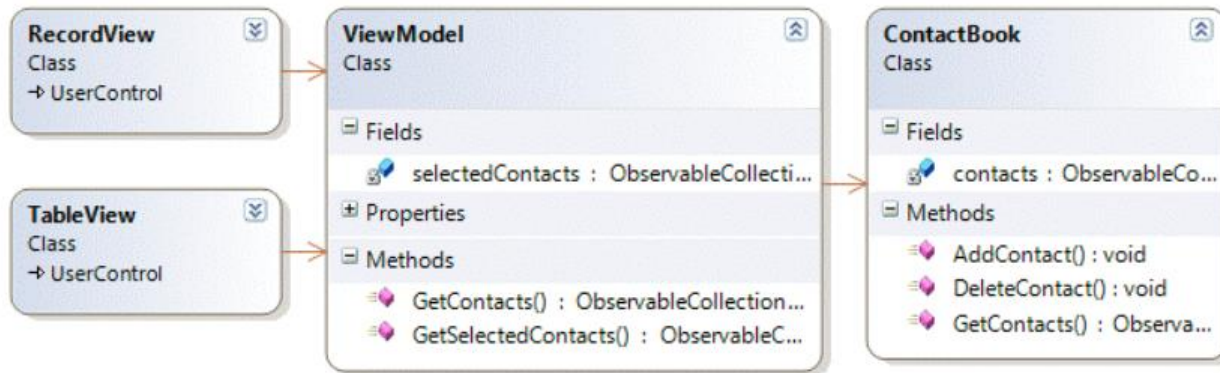
1. Track the selected item, so that we update RecordView whenever the selection in TableView changes, and vice versa.
2. Enable or disable parts of the UI of RecordView and TableView based on some rule (for example, highlight any entry that has an e-mail in the live.com domain).

Databinding is the ability to bind UI elements to any data.

Commands provide the ability to notify the underlying data of changes in the UI.

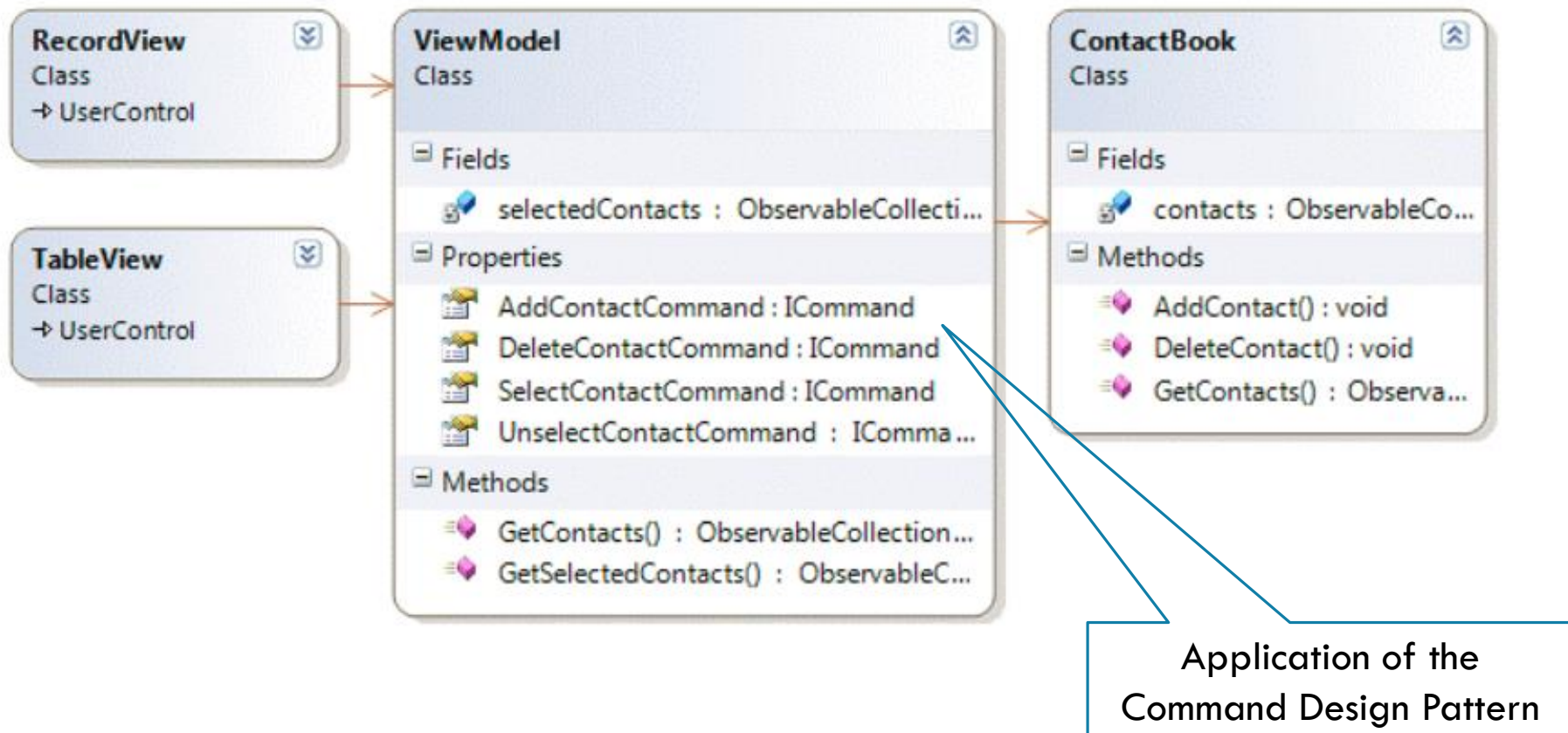


MVVM DESIGN



- The views know of the ViewModel and bind to its data, to be able to reflect any changes in it.
- The ViewModel has no reference to the views—it holds only a reference to the model.
- For the views, the ViewModel acts both as
 - a façade to the model,
 - a way to share state between views (selectedContacts in the example).
- The ViewModel often exposes commands that the views can bind to and trigger.

COMMANDS IN MVVM APPS



MVVM

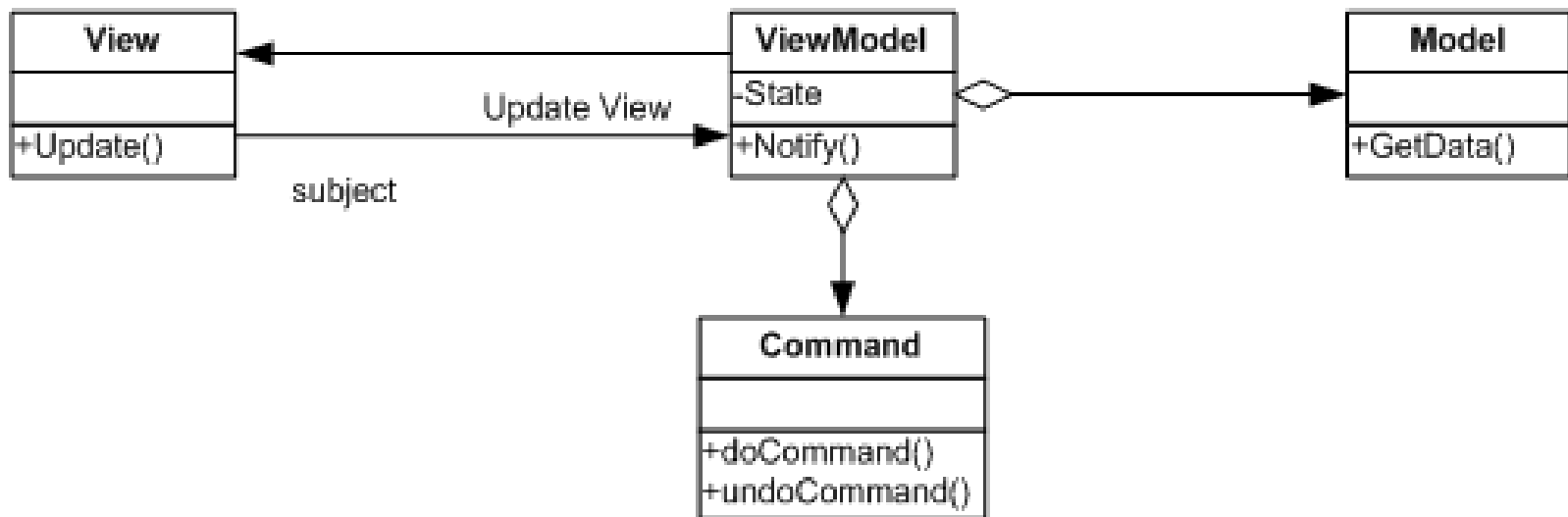
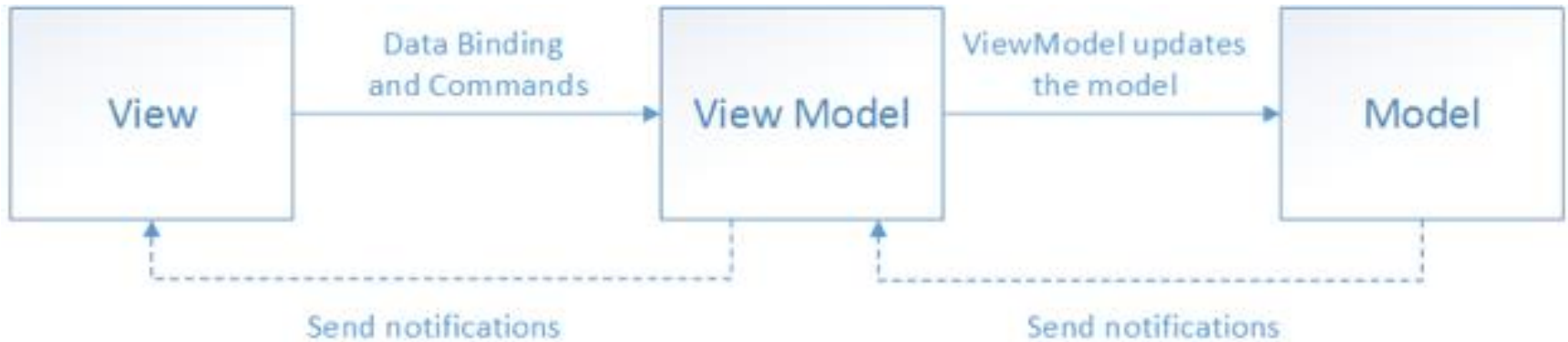


Figure 8: MVVM

MVVM STRUCTURE



Model

- Represents the data
 - 2 perspectives
 - OO approach: Domain Objects
 - Data-centric approach: XML, DAL, etc.
- No knowledge of how and where it will be presented
- Implements `INotifyPropertyChanged`

MVVM STRUCTURE

ViewModel

- Contains Properties, Commands, etc. to facilitate communication between View and Model
- Properties that expose instances of Model objects
- Commands and events that interact with Views for user interactions
- Implements INotifyPropertyChanged
- Pushes data up to the View

View

- Visual display
- Buttons, windows, graphics, etc.
- Responsible for displaying data
- Not responsible for retrieving data, business logic, business rules or validation
- Contains binding extensions
 - Identify data points represented to the user
 - Point to the names of the data point properties

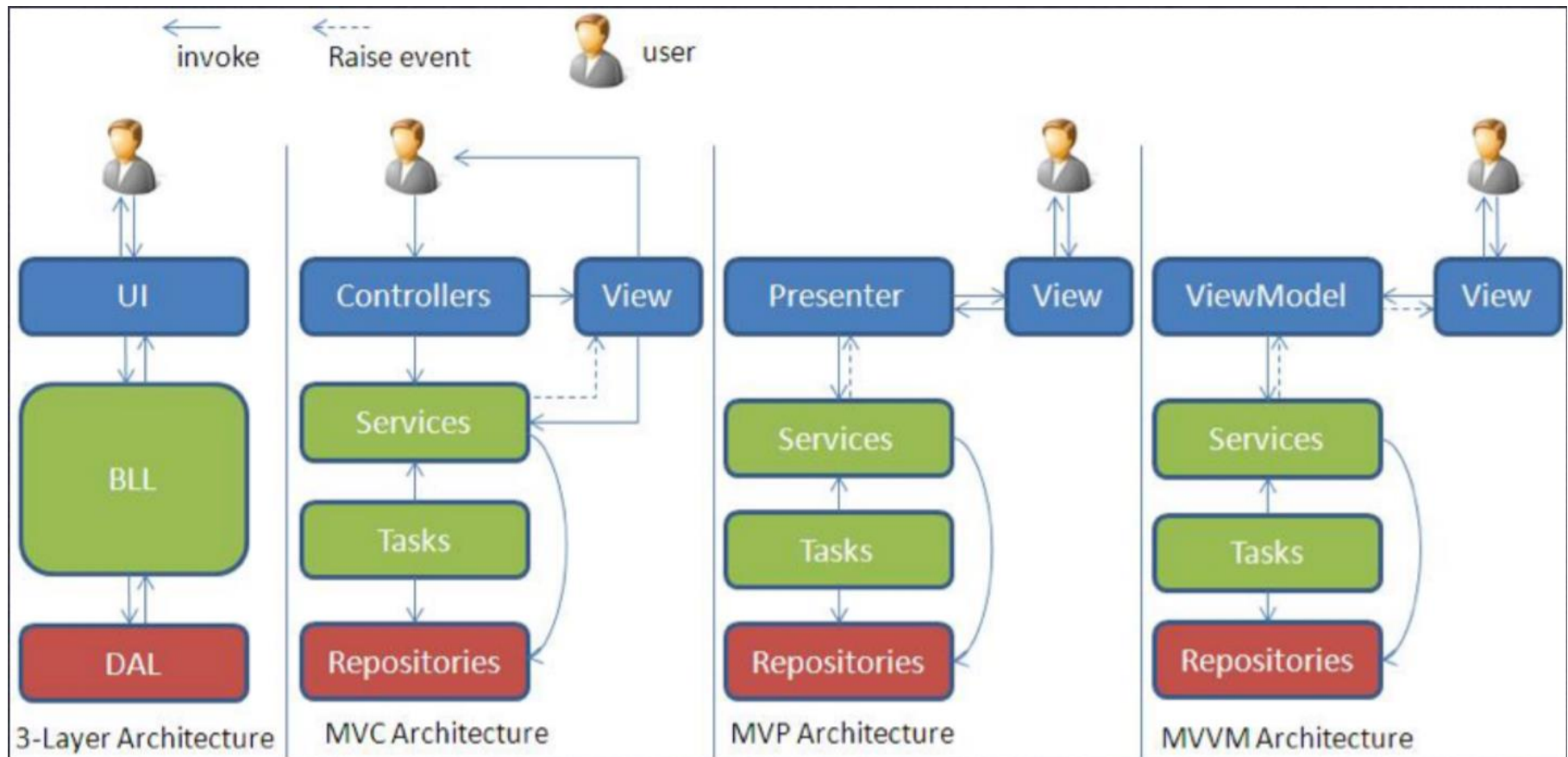
MVVM DISCUSSION

- MVVM is a derivative of MVC that takes advantage of particular strengths of the Windows Presentation Foundation (WPF) architecture
- Separates the Model and the View by introducing an abstract layer between them: a “View of the Model,” or ViewModel.
- The typical relationship between a ViewModel and the corresponding Views is one-to-many, but not always.
- There are situations when one ViewModel is aware of another ViewModel within the same application. When this happens, one ViewModel can represent a collection of ViewModels.

MVVM BENEFITS

- A ViewModel provides a **single store for presentation** policy and state, thus improving the reusability of the Model (by decoupling it from the Views) and the replaceability of the Views (by removing specific presentation policy from them).
- MVVM improves the overall **testability** of the application.
- MVVM also improves the “**mockability**” of the application.
- MVVM is a very **loosely coupled** design. The View holds a reference to the ViewModel, and the ViewModel holds a reference to the Model. The rest is done by the data-binding and commanding infrastructure of WPF.

MV* IN A NUTSHELL



WRAP-UP

Architectural decisions:

- Monolithic vs. distributed

Fundamental architectural styles

- Layered
 - MVC (and variants)
- Pipeline

LET'S SWITCH TO MOODLE AND TAKE A QUIZ

It takes 4 minutes.